ALGORITHMS

Smart Hospital Management System Using Doubly Linked List and Queue in C++

Algorithm 1: Main Algorithm

1. Start.
2. Initialize the doubly linked list with **head = NULL**.
3. Initialize the FIFO queue with **front = NULL** and **rear = NULL**.
4. Display the main menu and accept the user's choice.
5. If choice is 1, call Register Patient.
6. If choice is 2, call Display Patient Records.
7. If choice is 3, call Search Patient by Token.
8. If choice is 4, call Call Next Patient.
9. If choice is 5, call Count Waiting Patients and Estimate Waiting Time.
10. If choice is 6, call Discharge Patient.
11. If choice is 7, call View Dashboard.
12. If choice is 8, release allocated memory and terminate the program.
13. For choices 1–7, return to the main menu after completing the operation.
14. Stop.

Algorithm 2: Register Patient

1. Start.
2. Enter the patient's name, age, department, and symptoms.
3. Generate a unique token number.
4. Create a new patient node and store the patient details and token number.
5. Insert the node at the end of the doubly linked list.
6. Update the **prev** and **next** pointers.
7. Insert the token number into the FIFO waiting queue.
8. Display the registration confirmation and token number.
9. Return to the main menu.

Algorithm 3: Display Patient Records

1. Start.
2. Check whether **head == NULL**.
3. If true, display “No patient records found” and return to the main menu.
4. Otherwise, set **temp = head**.
5. Display the details of the current patient.
6. Move to the next node using **temp = temp->next**.
7. Repeat until **temp == NULL**.
8. Return to the main menu.

Algorithm 4: Search Patient by Token

1. Start.
2. Enter the token number to search.
3. Set **temp = head**.
4. Compare the token number with the current patient's token.
5. If a match is found, display the patient details.
6. Otherwise, move to the next node and repeat the comparison.
7. If no match is found by the end of the list, display “Patient not found.”
8. Return to the main menu.

Algorithm 5: Call Next Patient

1. Start.
2. Check whether the FIFO waiting queue is empty.
3. If empty, display “No patients waiting.”
4. Otherwise, remove the token from the front of the queue.
5. Search for the corresponding patient record.
6. Display the patient's token number, name, and department.
7. Update the patient's status to “In Consultation.”
8. Return to the main menu.

Algorithm 6: Count Waiting Patients and Estimate Waiting Time

1. Start.
2. Initialize the waiting counter to zero.
3. Traverse the FIFO queue and count the waiting patients.
4. Determine the number of patients ahead of the next patient.
5. Define the average consultation time per patient.
6. Calculate estimated waiting time: **Number of patients ahead × Average consultation time**.
7. Display the waiting count and estimated waiting time.
8. Return to the main menu.

Algorithm 7: Discharge Patient

1. Start.
2. Enter the patient's token number.
3. Search for the patient in the doubly linked list.
4. If the patient is not found, display “Patient not found.”
5. Otherwise, update the patient's status to “Discharged.”
6. Remove the patient from the waiting queue if the token is still present.
7. If the implementation deletes discharged records, update neighboring nodes' **prev** and **next** pointers.
8. Release the node's memory only if the record is deleted.

9. Display the discharge confirmation message.

10. Return to the main menu.

Algorithm 8: View Dashboard

1. Start.

2. Initialize counters for registered, waiting, in-consultation, and discharged patients.

3. Traverse the patient list and count patients according to their status.

4. Count the patients in the FIFO queue.

5. Display total registered patients.

6. Display waiting, in-consultation, and discharged patient counts.

7. Display department-wise statistics if implemented.

8. Return to the main menu.

Algorithm 9: Exit

1. Start.

2. Release all dynamically allocated patient nodes.

3. Release any remaining queue nodes, if dynamically allocated.

4. Set the pointers to **NULL**.

5. Display the exit message.

6. Stop.

Implementation note: Keep the patient list and waiting queue synchronized. Whether discharged records are retained or deleted should match the actual C++ program.